

Lab 05

OpenGL Modeling Transformations, OpenGL Quadrics

Objectives

- We will learn the basic of modelling **transformations** in this lab.
- We will also learn to model using **quadrics** in OpenGL.

Special Instructions

READ these lab instructions BEFORE starting!

1. By now, you should know that you need to create the new files – CGLab05.hpp and CGLab05.cpp. You are also expected to know how to create new classes and their member functions, define and call them appropriately. Please refer to **Task 0, Lab04** if not sure!
2. DO NOT copy-and-paste blindly! Some portions of code provided are just modifications, or additions of some lines of code. Always check carefully WHAT and WHERE to copy and paste. Most modifications/additions are in **BOLD**.

Programming Exercises

(80 minutes)

Task 1 : Using Modeling Transformation (70 minutes)

To start our exploration on transformation, first we create a class called `MyArrows` which contains a member function `drawArrow()`. Its implementation code is as given below:

```
void MyArrows::drawArrow()
{
    glBegin(GL_TRIANGLES);
        glVertex3f( 0.0f, 15.0f, 0.0f);
        glVertex3f(-2.0f, 13.0f, 0.0f);
        glVertex3f( 2.0f, 13.0f, 0.0f);
    glEnd();
    glBegin(GL_QUADS);
        glVertex3f(-1.0f, 13.0f, 0.0f);
        glVertex3f(-1.0f,  0.0f, 0.0f);
        glVertex3f( 1.0f,  0.0f, 0.0f);
        glVertex3f( 1.0f, 13.0f, 0.0f);
    glEnd();
}
```

Take note that this function will draw one arrow figure. We will create more drawing functions that will make use of this `drawArrow()` function to draw many arrows.

Task 1A: Understanding Translation (10 minutes)

Consider the code below:

```
void MyArrows::draw1()
{
    glDisable(GL_CULL_FACE);

    glColor3f(1.0, 0.0, 0.0);
    drawArrow();

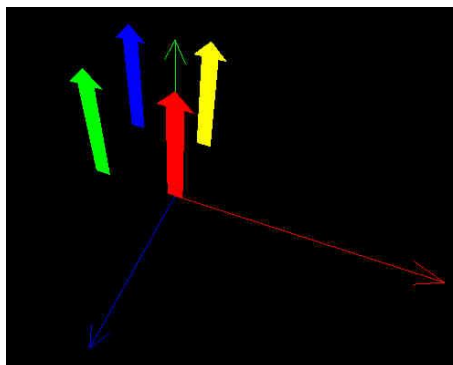
    glTranslatef(-10.0f, 0.0f, 0.0f);
    glColor3f(0.0, 1.0, 0.0);
    drawArrow();

    glTranslatef( 0.0f, 0.0f,-10.0f);
    glColor3f(0.0, 0.0, 1.0);
    drawArrow();

    glTranslatef( 10.0f, 0.0f, 0.0f);
    glColor3f(1.0, 1.0, 0.0);
    drawArrow();

    glEnable(GL_CULL_FACE);
}
```

The output would be:



Observe where the arrows have been moved. What is a **TRANSLATION**?

Take note that all the translations are cumulative, meaning that any translations (or other transformations) will affect subsequent translations (or other transformations) as well. Another way to look at it is that OpenGL keeps track of the current position and orientation. Any transformation that follows would be done relative to this current position. More accurately, any transformations (translation, rotation and scaling) will affect the **MODELVIEW matrix**. You may not have learnt this yet, but it is easy to just think now of MODELVIEW matrix as a matrix that keeps track of the current position, orientation and scaling factor. Any subsequent transformation will update the MODELVIEW matrix, and that in turn, modifies the current position.

Task 1B: Storing and Restoring Current Position and Orientation (10 minutes)

Consider the program below that is very similar to the above, except for the highlighted part:

```
void MyArrows::draw2()
{
    glDisable(GL_CULL_FACE);

    glColor3f(1.0, 0.0, 0.0);
    drawArrow();

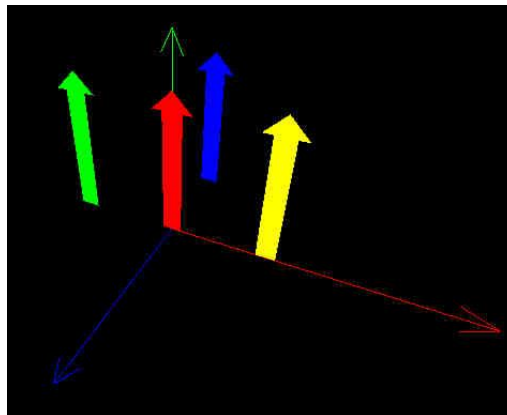
    glPushMatrix();
    glTranslatef(-10.0f, 0.0f, 0.0f);
    glColor3f(0.0, 1.0, 0.0);
    drawArrow();
    glPopMatrix();

    glPushMatrix();
    glTranslatef( 0.0f, 0.0f,-10.0f);
    glColor3f(0.0, 0.0, 1.0);
    drawArrow();
    glPopMatrix();

    glPushMatrix();
    glTranslatef( 10.0f, 0.0f, 0.0f);
    glColor3f(1.0, 1.0, 0.0);
    drawArrow();
    glPopMatrix();

    glEnable(GL_CULL_FACE);
}
```

The output would be:



Each portion of translation is now enclosed within a pair of `glPushMatrix()` and `glPopMatrix()` functions.

Discuss what is the difference between this program and the program earlier.

Why the output is different?

HINT: `glPushMatrix()` actually saves the current MODELVIEW matrix and `glPopMatrix()` actually restore the previously-saved matrix. More on this in later labs.

Task 1C: Understanding Rotation (10 minutes)

Consider the code below:

```
void MyArrows::draw3()
{
    glDisable(GL_CULL_FACE);

    glColor3f(1.0, 0.0, 0.0);
    drawArrow();

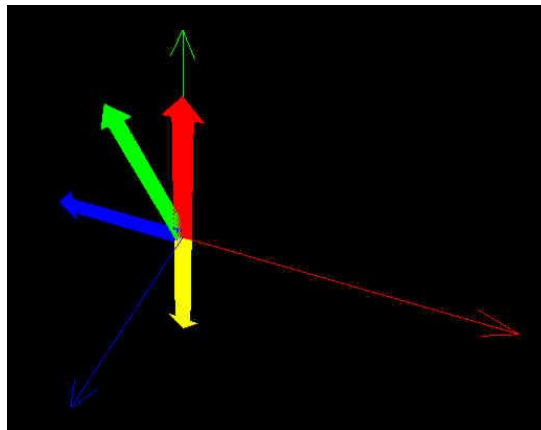
    glRotatef(30.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(0.0, 1.0, 0.0);
    drawArrow();

    glRotatef(60.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(0.0, 0.0, 1.0);
    drawArrow();

    glRotatef(90.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(1.0, 1.0, 0.0);
    drawArrow();

    glEnable(GL_CULL_FACE);
}
```

The output will be:



What is a **ROTATION**?

Now, try this code (with `glPushMatrix()` and `glPopMatrix()` functions) as well:

```
void MyArrows::draw4()
{
    glDisable(GL_CULL_FACE);

    glColor3f(1.0, 0.0, 0.0);
    drawArrow();

    glPushMatrix();
    glRotatef(30.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(0.0, 1.0, 0.0);
    drawArrow();
    glPopMatrix();
}
```

```

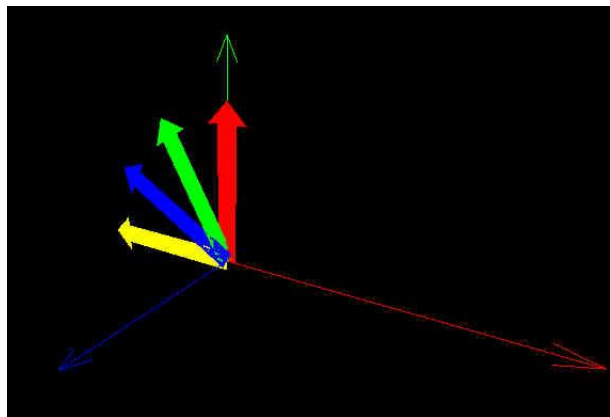
glPushMatrix();
    glRotatef(60.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(0.0, 0.0, 1.0);
    drawArrow();
glPopMatrix();

glPushMatrix();
    glRotatef(90.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(1.0, 1.0, 0.0);
    drawArrow();
glPopMatrix();

    glEnable(GL_CULL_FACE);
}

```

The output will be:



Again, discuss the difference between the output of draw3() and draw4()?

Can you make sense of the usage of glPushMatrix() and glPopMatrix() functions now?

Task 1D: Understanding Transformation Concatenation (10 minutes)

Consider the following code, with Translation before Rotation:

```

void MyArrows::draw5()
{
    glDisable(GL_CULL_FACE);

    glColor3f(1.0, 0.0, 0.0);
    drawArrow();

    glPushMatrix();
        glTranslatef(-10.0f, 0.0f, 0.0f);
        glRotatef(30.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(0.0, 1.0, 0.0);
        drawArrow();
    glPopMatrix();

    glPushMatrix();
        glTranslatef( 0.0f, 0.0f,-10.0f);
        glRotatef(60.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(0.0, 0.0, 1.0);

```

```

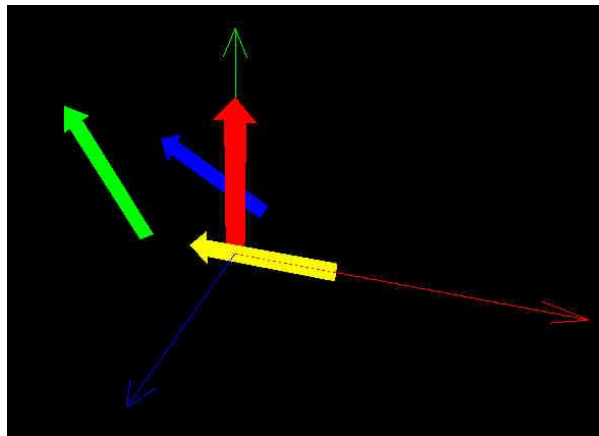
    drawArrow();
    glPopMatrix();

    glPushMatrix();
        glTranslatef( 10.0f, 0.0f, 0.0f);
        glRotatef(90.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(1.0, 1.0, 0.0);
        drawArrow();
    glPopMatrix();

    glEnable(GL_CULL_FACE);
}

```

The output:



The program above contains the transformation `glTranslatef(...)` followed by `glRotatef(...)`. Can you make sense of the above output?

IMPORTANT: You should think in terms of transformation of coordinate system rather than transformation of objects!!

If you're still not clear about it, discuss and compare with the code below, where we swap the sequence of transformation. Now, Rotation before Translation:

```

void MyArrows::draw6()
{
    glDisable(GL_CULL_FACE);

    glColor3f(1.0, 0.0, 0.0);
    drawArrow();

    glPushMatrix();
        glRotatef(30.0f, 0.0f, 0.0f, 1.0f);
        glTranslatef(-10.0f, 0.0f, 0.0f);
        glColor3f(0.0, 1.0, 0.0);
        drawArrow();
    glPopMatrix();

    glPushMatrix();
        glRotatef(60.0f, 0.0f, 0.0f, 1.0f);
        glTranslatef( 0.0f, 0.0f, -10.0f);
        glColor3f(0.0, 0.0, 1.0);
        drawArrow();
    glPopMatrix();
}

```

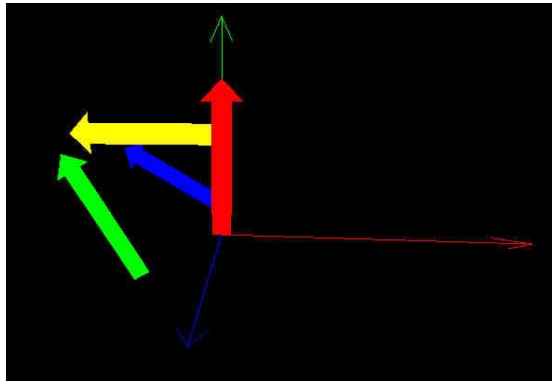
```

glPushMatrix();
    glRotatef(90.0f, 0.0f, 0.0f, 1.0f);
    glTranslatef( 10.0f, 0.0f, 0.0f);
    glColor3f(1.0, 1.0, 0.0);
    drawArrow();
glPopMatrix();

glEnable(GL_CULL_FACE);
}

```

The output is now:

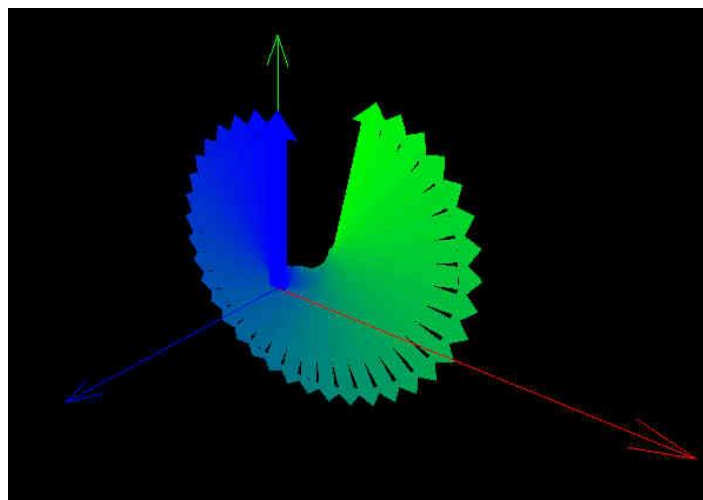


Ah! Discuss why the above two sets of code produce output that are so different?

Understanding this will help you to plan and code your transformations correctly in future exercises and also for your project work.

Task 1E: Putting it together - Exercise 1 (15 minutes)

Write a new void `MyArrows::draw7()` function to draw the figure below:



HINT: You need to gradually increase the angle of rotation of the arrow, and also gradually change the colour of the arrow from blue to green! And, all this can be easily accomplished with some useful loops.

Task 2 : Using Quadrics (20 minutes)**Task 2A: Understanding Quadrics**

The code below demonstrates how to use Quadrics:

```
void MyQuadricDemo::draw()
{
    glDisable(GL_CULL_FACE);

    //Quadric Object
    GLUQuadricObj *pObj;

    //create and initialize quadric
    pObj = gluNewQuadric();

    //try one of these drawing style
    //gluQuadricDrawStyle(pObj, GLU_FILL);
    //gluQuadricDrawStyle(pObj, GLU_LINE);
    //gluQuadricDrawStyle(pObj, GLU_POINT);
    //gluQuadricDrawStyle(pObj, GLU_SILHOUETTE);

    //try one of these surface normals mode
    //gluQuadricNormals(pObj, GLU_NONE);
    gluQuadricNormals(pObj, GLU_SMOOTH);
    //gluQuadricNormals(pObj, GLU_FLAT);

    glPushMatrix();
        glColor3f(1.0f, 1.0f, 1.0f);
        glTranslatef(-40.0f, 0.0f, 0.0f);
        gluCylinder(pObj, 2.0f, 12.0f, 20.0f, 24, 72);

        glColor3f(1.0f, 1.0f, 0.0f);
        glTranslatef(20.0f, 0.0f, 0.0f);
        gluCylinder(pObj, 2.0f, 12.0f, 20.0f, 72, 4);

        glColor3f(0.0f, 1.0f, 0.0f);
        glTranslatef(14.0f, 0.0f, 0.0f);
        gluSphere(pObj, 8.0f, 24, 12);

        glColor3f(0.0f, 0.0f, 1.0f);
        glTranslatef(14.0f, 0.0f, 0.0f);
        gluSphere(pObj, 4.0f, 24, 12);

        glColor3f(1.0f, 0.0f, 0.0f);
        glTranslatef(14.0f, 0.0f, 0.0f);
        gluDisk(pObj, 4.0f, 10.0f, 26, 13);

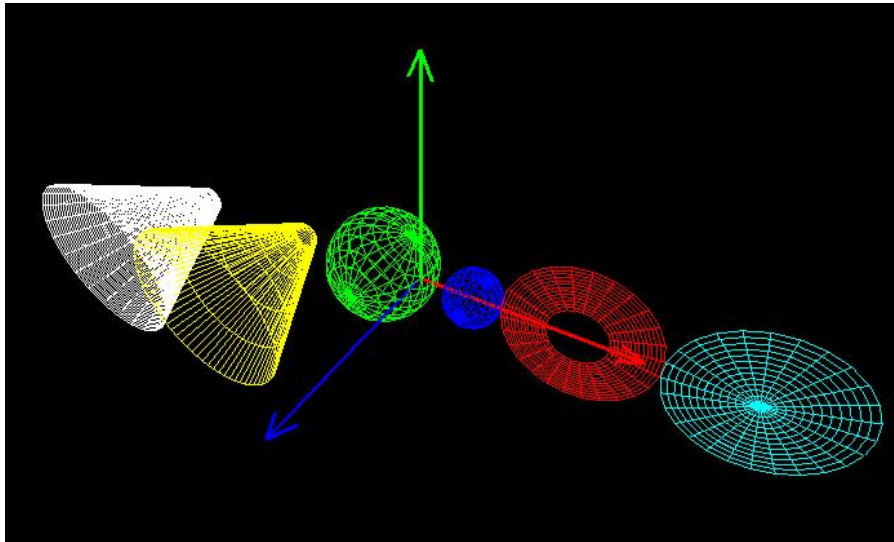
        glColor3f(0.0f, 1.0f, 1.0f);
        glTranslatef(20.0f, 0.0f, 0.0f);
        gluDisk(pObj, 0.0f, 10.0f, 26, 13);

    glPopMatrix();

    gluDeleteQuadric(pObj);           //free the quadric object

    glEnable(GL_CULL_FACE);
}
```

The output is as below (drawn in wire-frame mode, press F1 to toggle this mode):



OK, what are **QUADRICS**? What are the different types of “quadrics”?

Try to understand this code by experimenting with the parameters used to call the `gluCylinder()`, `gluSphere()` and `gluDisk()` functions.

Take note that the quadrics functions are from the “glu” library rather than the core “gl” library. This is because these functions are utility functions that convert the quadrics into the most basic OpenGL primitives such as lines, points, quads, triangles or polygons. In other words, these quadric functions help you to draw more complex primitives such as cylinders, cones, spheres, discs and hollow discs, in a much easier way. Would you want to build these object shapes by just lines?

The prototype of these quadric functions are as given below:

- `void gluCylinder (GLUquadricObj *qobj, GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices, GLint stacks);`
- `void gluDisk (GLUquadricObj *qobj, GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint loops);`
- `void gluSphere (GLUquadricObj *qobj, GLdouble radius, GLint slices, GLint stacks);`

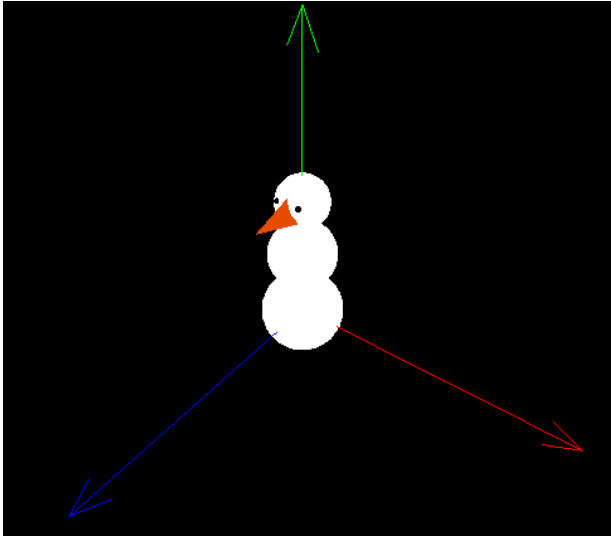
By experimenting with the many parameters above, you should try to understand each of these parameters so that you are able to create a variety of different quadrics. For example, the higher the number for `slices`, `stacks` and `loops`, the smoother the model will be. What do these parameters represent?

HINT: Trying to understand all parameters of a complex OpenGL function, such as `gluCylinder` can be a daunting task. Look up the OpenGL Reference Guide (available online) for the function documentation. Search for the function, and read the description given for each parameter. It helps!

IMPORTANT: Take note of the default orientation of the quadric objects. If you wish to have a different orientation, then you should rotate them accordingly, as already learned in **Task 1**.

Task 2B: Applying Quadrics

Modify the program above to draw a cute Snowman as shown below, simply just using quadrics!



HAVE FUN !!

* END OF LAB *